DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Report

Threat Modelling in Large Language Models

By Nguyen Huy Hai

Ha Noi, August 2026

Chapter 1

Introduction

Threat modeling is not an isolated preliminary step in this project, the organizing method that many component builds on.

- **Intent Drift identification:** Threat modeling turns Intent Drift from a vague phenomenon into a defined taxonomy of attack vectors, a systematic account of how and where drift occurs.
- **Red Teaming:** That taxonomy defines the attack tree the Red Teaming AI searches, so the framework generates targeted adversarial test cases instead of relying on random prompt fuzzing.
- **Monitoring and defenses:** The same threat model specifies the deviation signatures, what drift looks like mechanistically, semantic integrity monitoring is built to detect, and that intent-aware defenses act on.
- **Multilingual/cross-cultural benchmarks:** Threat modeling identifies which attack patterns are genuinely language and culture-specific such as idiom, script-level tricks, culturally specific social-engineering. So the benchmark tests these directly rather than relying on English-language attacks that have simply been translated.

Together, these roles make threat modeling the shared reference model that unifies Red Teaming, monitoring, provenance, verification, and benchmarking into defense pipeline.

Chapter 2

Related Works

ThreMoLIA — Threat Modeling of Large Language Model Integrated Applications

- **Problem/gap:** Threat modeling for LLM-integrated applications (LIAs) is manual and effort-intensive; no automated support exists.
- **Approach:** LLM + RAG-assisted framework that starts from a system Data Flow Diagram and combines MITRE ATLAS, OWASP Top 10 for LLMs, and STRIDE to generate/update a threat model, augmented with retrieval over existing threat models, security knowledge, and architecture docs.
- **Key result(s):** Proposes a "health score" concept (19 candidate metrics — e.g., threat coverage, residual risk, model complexity) to estimate threat-model completeness/quality; no concrete scoring formula defined yet.
- **Limitation:** Early-stage vision paper; evaluation limited to a single proof-of-concept case study on ChatGPT-3.5; no formula validated with practitioners; no industrial case studies.
- **Relevance to this work:** [**ASSUMPTION — confirm/edit**] ThreMoLIA's LLM+RAG-assisted, DFD-driven approach to generating a threat model is the closest prior work to the threat-modeling stage of this framework, but lacks the intent-drift-specific taxonomy and downstream automation (Red Teaming, monitoring, provenance) this work builds on top of a threat model.

Red Teaming Language Models with Language Models (Perez et al., EMNLP 2022)

- **Problem/gap:** Manual Red Teaming (humans hand-writing adversarial prompts) is costly and doesn't scale.
- **Approach:** A "red LM" automatically generates adversarial test inputs in a three-stage pipeline: (1) red LM generates adversarial test input x, (2) target LM produces reply y, (3) a classifier $r(x, y)$ flags harmful outputs. Compares four generation methods (zero-shot, stochastic few-shot, supervised learning, RL) on a diversity/difficulty trade-off.

- **Key result(s):** RL is most effective at eliciting harm but prone to mode collapse at aggressive settings (78% of generated questions converged on "invisible"). Applied to a 280B-parameter chatbot, surfaced five harm categories: offensive replies, training-data leakage, generated contact info, distributional bias, and multi-turn compounding harms.
- **Limitation:** Effectiveness is bounded by the red LM and classifier, both of which inherit biases from their training data; complements rather than replaces manual Red Teaming.
- **Relevance to this work:** [**ASSUMPTION**] Using an LM to automate adversarial test generation, directly relevant to the AI-driven Red Teaming component of this framework; this work's threat model supplies that structure to target generation rather than searching unconstrained.

HarmBench — A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal

- **Problem/gap:** Prior automated Red Teaming evaluations lack breadth (most test < 100 behaviors), comparability (e.g., completion length alone can swing ASR by up to 30%), and robust classifiers (many fail simple adversarial sanity checks, such as missing a model that initially refuses but then complies anyway).
- **Approach:** 510 curated harmful behaviors across 7 semantic categories and 4 functional categories, with an official validation/test split. Pipeline: generate test cases $\rightarrow$ generate completions (fixed 512 tokens, greedy decoding) $\rightarrow$ classify success via a fine-tuned classifier. Also introduces R2D2, an adversarial-training defense method. Recheck with (As an AI, i am sorry, ...) and perplexity, not just ASR.
- **Key result(s):** Benchmarks 18 Red Teaming methods against 33 target LLMs/defenses; finds no attack or defense is uniformly effective, and robustness does not scale with model size within a family — training data/procedure matters more. R2D2 achieves state-of-the-art robustness against GCG while preserving general capability.
- **Limitation:** Standardization is scoped to the specific behavior set and classifiers provided; not stated to cover multilingual or culturally-specific attack behaviors.
- **Relevance to this work:** [**ASSUMPTION**] HarmBench's emphasis on standardized, comparable evaluation is the model for this work's own benchmarking approach, but its behavior set is not designed for the language and culture-specific attack patterns this work's multilingual benchmarks target.

MITRE ATLAS

- **Problem/gap:** Shared vocabulary existed for cataloging real-world adversarial attacks against AI/ML systems.
- **Approach:** A living knowledge base structured like MITRE ATT&CK, organizing attacks into tactics (adversary objectives) and techniques (specific methods), each with identifiers, descriptions, mitigations, and references to documented incidents.

- **Key result(s):** Continuously updated; recent releases expand coverage of generative-AI threats such as prompt injection and RAG-related attacks.
- **Limitation:** Not stated in the source material beyond general scope (curated/manual maintenance; framework-level rather than automated).
- **Relevance to this work:** [**ASSUMPTION**] ATLAS serves as the technique-level checklist for this work's threat register, mapping identified Intent Drift threats to ATLAS techniques gives a standardized, cross-referenceable vocabulary.

Azure AI Content Safety Azure AI Content Safety is Microsoft's production content-moderation service, offered as part of Azure AI Foundry, for detecting harmful content in both user-generated and AI-generated text and images.

- **Problem/gap:** Deployed LLM applications need a way to detect and grade harmful content in real time, not just flag it as present or absent.
- **Approach:** An ensemble of classification models scores text (and images) against four harm categories: Hate/Fairness, Sexual, Violence, and Self-Harm. Each on an internal 0-7 severity scale. The API can also return a trimmed 4-level scale (0, 2, 4, 6), where adjacent raw levels are collapsed together, letting applications set per-category severity thresholds rather than a single pass/fail cutoff. The broader service also includes Prompt Shields for prompt-injection detection and groundedness detection for hallucination detection.
- **Key result(s):** Provides configurable, per-category severity thresholds integrated natively into Azure OpenAI Service, so prompts and completions are filtered automatically without additional application logic.
- **Limitation:** Categories are fixed to general content-harm types (hate, sexual, violence, self-harm) rather than security-specific attack vectors; it is not designed to classify Intent Drift or other reasoning-level security threats.
- **Relevance to this work:** [ASSUMPTION] Azure AI Content Safety's graded severity model (fine-grained internal scoring, coarser reported levels) is a useful industry reference for grading the severity of detected Intent Drift events, but its fixed harm categories do not map onto this work's threat taxonomy, motivating a drift-specific severity scale.

Chapter 3

Methodology - Planning

3.1 Define the System Boundary

Threat modelling starts with a DFD(Data Flow Diagram), then STRIDE is applied systematically to each component to identify: external entities, processes, databases, trust boundaries, and data flows. After that, the security bug hunt starts.

3.2 Enumerate Threats with STRIDE

STRIDE is a threat modeling framework developed by Microsoft that categorizes security threats into six classes: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. For LLM-integrated applications, these categories can be reinterpreted as security questions that guide the identification of AI-specific threats, such as prompt injection, hidden prompt leakage, and context exhaustion.

Table 3.1: Interpreting STRIDE for LLM Threat Modeling

Threat	**Question**	**LLM Interpretation Example**
Spoofing	Can someone pretend to be someone else?	Can someone impersonate the LLM provider?
Tampering	Can someone modify data or code?	Can prompts be altered?
Repudiation	Can someone deny performing an action?	Can prompt history be deleted?
Information Disclosure	Can confidential information leak?	Can hidden prompts leak?
Denial of Service	Can the system be made unavailable?	Can huge prompts exhaust context?
Elevation of Privilege	Can someone gain more permissions than intended?	Can prompt injection bypass safety instructions?

Here there are some threat model matrix for LLM/LMM:

Table 3.2: Example Threat Model Matrix for LLM/LMM

Threat	**STRIDE Category**	**OWASP Mapping**	**Researched Mitigations**
Direct Prompt Injection	Tampering / Elevation of Privilege	LLM01: Prompt Injection	Dual-LLM/LMM verification architectures, Strict separation of data and instructions, Clear input boundaries
Indirect Prompt Injection	Tampering	LLM01: Prompt Injection	Isolated sandbox parsing for untrusted data (web pages, PDFs), Semantic boundary sanitization before the LLMs receive an input
Cross-Modal Injection (Pixel/Audio Jailbreaks)	Tampering / Spoofing	LLM01: Prompt Injection (Multimodal Extension)	OCR pre-scans on images, Adversarial perturbation filtering, Vision/audio encoder alignment checks
Jailbreaking / Guardrail Bypass	Elevation of Privilege	LLM01: Prompt Injection	Layered system instructions, Independent guardrail models (e.g., Llama Guard), Defensive alignment tuning (RLHF/DPO)
System Prompt Leakage	Information Disclosure	LLM02: Sensitive Information Disclosure	Strict prompt isolation, Programmatic output regex/semantic filtering, Structural design preventing system prompt disclosure
Multimodal Data Poisoning (Training/Fine-tuning)	Tampering	LLM03: Training Data Poisoning	Cryptographic provenance tracking for training data and model weights, Robust data-cleaning pipelines, Gradient anomaly detection

3.3 Score and Prioritize with DREAD

Each threat is scored across five dimensions: Damage, Reproducibility, Exploitability, Affected Users, Discoverability.

Table 3.3: MITRE ATLAS Checklist - Threat Register

Threat	**STRIDE Component**	**Asset Targeted**	**Trust Boundary**	**ATLAS Tactic**	**ATLAS Technique**
Direct Prompt Injection	Tampering	System Prompt & Intent Parser	User → Orchestrator	Initial Access / Execution	AML.T0051: LLM Prompt Injection
Behavioral Jailbreaking (e.g., Crescendo)	Elevation of Privilege	Safety Alignment Weights & Policies	User → LLM Core Engine	Privilege Escalation / Defense Evasion	AML.T0054: LLM Jailbreak
System Prompt Leakage	Information Disclosure	Hidden Architecture / Context Strings	Core Engine → User	Collection / Exfiltration	AML.T0056: LLM Meta Prompt Extraction
Resource Exhaustion (Token/Context Flooding)	Denial of Service	Compute Allocation & Memory Buffers	User → Gateway	Impact	AML.T0029: Denial of ML Service
Indirect RAG Injection (Retrieval Poisoning)	Tampering	Vector Knowledge Base Embeddings	External DB → Orchestrator	Initial Access / Execution	AML.T0051.001: Indirect Prompt Injection
Cross-Modal Exploitation (Pixel-Hidden Injection)	Spoofing / Tampering	Vision/Audio Multimodal Encoders	External Media Input → LMM	ML Attack Staging / Initial Access	AML.T0043: Craft Adversarial Data

3.4 Attack Taxonomy

Table 3.4: Attack Taxonomy for LLM/LMM Threats

Category	**STRIDE**	**Example Threats**	**Example Mitigations**
Prompt Injection	Tampering, Elevation of Privilege	Direct, indirect, cross-agent injection	Prompt isolation, input validation, tool authorization
Adversarial Manipulation	Tampering	Data poisoning, retrieval poisoning, adversarial examples	Data validation, provenance checks, retrieval filtering
Hallucination Exploitation	Information Disclosure, integrity-related impacts	Fabricated citations, false APIs, unsafe advice	Grounding, confidence estimation, human review
Semantic Misalignment	Elevation of Privilege, Tampering	Goal manipulation, instruction conflicts	Alignment testing, policy constraints, evaluation suites
Cross-modal Attacks	Tampering	OCR injection, hidden text, image-based prompts	Content sanitization, modality-specific filtering